

```
ghci> let xs = toS [1..5] :: Seq Int
ghci> xs
1 | 2 | 3 | 4 | 5 | ? .
ghci> delay xs
? | 1 | 2 | 3 | 4 | 5 | ? .
ghci> register 10 xs
10 | 1 | 2 | 3 | 4 | 5 | ? .
```

Kansas Lava has support for both fixed-width signed and unsigned numbers. An unsigned X4 is a 4-bit unsigned number.

```
ghci> :m + Data.Sized.Matrix
ghci> :m + Data.Sized.Unsigned
ghci> 100 :: U4
4
ghci> :m + Data.Sized.Signed
ghci> 100 :: S8
100
ghci> 100 + 100 :: S8
-56
ghci> [minBound..maxBound] :: [U4]
[0,1,2,3,4,5,6,7,8,9,10,11,12,13,14,15]
```

It also provides support for fixed-width matrices. A matrix with four elements of boolean type is represented by Matrix U4 bool. These are useful when specifying irregular-sized values in hardware.

```
ghci> :m + Data.Sized.Matrix
ghci> let m = matrix [1,2,3,4] ::
Matrix U2 Int
ghci> :m + Data.Array.IArray
ghci> m ! 1
2
ghci> m ! 0
1
```

The pack utility provides a means to combine multiple signals into a single signal, and the unpack combinator does the reverse. These allow flexible-type representation of data in the program. For example,

```
ghci> :m + Language.KansasLava
ghci> let p = toS (cycle [False, True])
:: Seq Bool
ghci> let q = toS [1..] :: Seq Int
ghci> let pq = pack (p, q) :: Seq (Bool,
Int)
```

```
ghci> takeS 2 pq
(low,1) | (high,2) | (? ,?) .
```

Apart from signal types, circuits can also be built from Haskell functions.

Consider a ROM that stores the cube of a value. funMapXXX function when used directly is similar to an asynchronous ROM.

```
ghci> :m + Language.KansasLava
ghci> :set -XScopedTypeVariables
ghci> let cubeROM = funMapXXX \(x::Int)
-> return(x*x*x)
ghci> takeS 5 $ cubeROM $ toS [1..] ::
Seq Int
1 | 8 | 27 | 64 | 125 | ? .
```

Kansas Lava also provides various coerce functions that can be used between two signal types, as long as both types have the same bit-width. For example,

```
ghci> :m + Language.KansasLava
ghci> let a = toS (cycle [False, True])
:: Seq Bool
ghci> takeS 5 a
low | high | low | high | low | ? .
ghci> takeS 5 $ (unsigned) a :: Seq Int
0 | 1 | 0 | 1 | 0 | ? .
```

Circuits can be debugged using probes, which present a way to observe intermediate values in shallow embedded circuits without modifying the circuit interface. These are inserted in the function using probeS function.

```
ghci> :m + Language.KansasLava
ghci> setProbesAsTrace (appendFile
"DEBUG")
ghci> let xs = toS (cycle [False, True])
:: Seq Bool
ghci> let ys = probeS "ys" (takeS 5 $
(unsigned) xs :: Seq Int)
ghci> takeS 5 ys
0 | 1 | 0 | 1 | 0 | ? .
ghci> :!cat DEBUG
ys(0) 0
ys(1) 1
ys(2) 0
ys(3) 1
ys(4) 0
```

An important purpose of Kansas Lava is to produce useful VHDL. Hardware descriptions are converted into efficient VHDL. Kansas Lava programs are extracted into a Kansas Lava entity graph (KLEG), which is an abstract representation of the netlist. This can be optimised and written into VHDL. The generic netlist rendering tool can also target Verilog. Kansas Lava can also generate and execute a VHDL test bench. mkTestbench function can be used to generate a test bench.

The code is listed below:

```
import Language.KansasLava
import Language.KansasLava.Fabric
import Language.KansasLava.VHDL

halfAdd :: Signal i Bool -> Signal i
Bool -> (Signal i Bool, Signal i Bool)
halfAdd a b = (carry, sum)
  where carry = a `and2` b
        sum = a `xor2` b

fabric :: Fabric ()
fabric = do
  i0 <- inStdLogic "i0"
  i1 <- inStdLogic "i1"
  let (c,s) = halfAdd i0 i1
  outStdLogic "carry" c
  outStdLogic "sum" s

main = do
  k <- reifyFabric fabric
  writeVhdlCircuit "halfAdd" "halfAdd.
vhd" k
  mkTestbench "halfAdd" ". " k
```

The above code can be compiled and run using:

```
$ ghc --make halfAdd.hs
$. /halfAdd
```

Following files are created in the directory where halfAdd is executed: halfAdd.vhd, halfAdd_tb.vhd and halfAdd.do. halfAdd.vhd contains the VHDL implementation of the circuit, halfAdd_tb.vhd contains testbench VHDL and halfAdd.do contains a ModelSim script to run the test bench.

halfAdd.vhd file is listed below:

```
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;
```